

4.5 TF-IDF

Down-weight words that appear in almost every document.

These notes walk through the lecture slides. The original deck is unchanged; use **(slides)** on the course hub if you want that PDF.

1. Not every word should count the same

One-hot, bag of words, and bag of n-grams treat every token as equally important. *the* and *pneumonia* get the same kind of count. That is a problem. A word that appears in almost every document does not tell you which document you are looking at.

TF-IDF (term frequency–inverse document frequency) reweights the counts. The claim: if word w is common in document d_i and rare in the rest of the corpus, then w is **about** d_i .

Two pieces multiply:

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t)$$

Same toy documents as the rest of Week 4. The encoding is still a $|V|$ -vector per document. The numbers are no longer raw counts.

2. Term frequency

TF asks how often term t appears **inside** one document. Longer documents have more room to repeat a word, so we normalize by document length:

$$\text{TF}(t, d) = \frac{\text{count of } t \text{ in } d}{\text{number of tokens in } d}$$

In $D_1 = \text{"dog bites man"}$, each of the three words has $\text{TF} = 1 / 3$.

Other textbooks use raw count, $\log(1 + \text{count})$, or a Boolean. The idea is the same: more mentions inside $d \rightarrow$ larger TF.

3. Inverse document frequency

IDF asks how special the term is **across the corpus**. Stop words (*is*, *are*, *the*) have high TF everywhere and should be down-weighted. Rare content words should be up-weighted.

$$\text{IDF}(t) = \ln \frac{N}{\text{number of documents that contain } t}$$

N is the number of documents. A word that appears in every document has $\text{IDF} = \ln 1 = 0$ (or a tiny number if the implementation adds smoothing). A word that appears in one document of four has IDF

$= \ln 4$.

Word	Docs that contain it	IDF $\ln(N / \text{df})$
dog	D_1, D_2, D_3	$\ln(4 / 3) \approx 0.29$
bites	D_1, D_2	$\ln(4 / 2) \approx 0.69$
man	D_1, D_2, D_4	$\ln(4 / 3) \approx 0.29$
eats	D_3, D_4	$\ln(4 / 2) \approx 0.69$
meat	D_3	$\ln(4 / 1) \approx 1.39$
food	D_4	$\ln(4 / 1) \approx 1.39$

meat and *food* are the most informative unigrams in this toy set. *dog* and *man* are closer to stop words.

4. The product, and sklearn

For $D_3 = \text{"dog eats meat"}$, TF of each word is $1 / 3$. The TF-IDF entries (in vocabulary order *dog*, *bites*, *man*, *eats*, *meat*, *food*) are about

```
[0.10, 0, 0, 0.23, 0.46, 0]
```

meat dominates. That is the scheme working.

Your hand calculation will not match `TfidfVectorizer`. sklearn uses a smoothed IDF, sometimes L2-normalizes the rows, and may use $\log(N / \text{df}) + 1$. Read the docstring before you grade a lab against a spreadsheet. Several published variants exist. The product $\text{TF} \times \text{IDF}$ is the shared idea.

5. What still fails

TF-IDF is still a **discrete** representation. *movie* and *film* do not share weight. There is no geometry that says they are related.

The vectors are still **sparse and high-dimensional**. Larger vocabulary $\rightarrow$ more zeros $\rightarrow$ harder learning and more RAM.

OOV is unchanged. A new word has no IDF because it has no document frequency in the training corpus.

Use TF-IDF as the default sparse baseline: `TfidfVectorizer` + linear SVM or logistic regression. Then ask whether a dense embedding (Week 5) is worth the complexity.

6. Practice

1. Compute TF-IDF for *bites* in D_1 . Use the formulas in sections 2 and 3.
2. Why does IDF push *the* toward zero on a news corpus even if *the* has a huge TF?

3. Name two reasons a TF-IDF vector for "the film was long" will not be close to one for "the movie was long."